

RAIDE: a tool for Assertion Roulette and Duplicate Assert identification and refactoring

Railana Santana
railana.santana@ufba.br
UFBA, Bahia, Brazil

Luana Martins
martins.luana@ufba.br
UFBA, Bahia, Brazil

Larissa Rocha
lrsoares@uefs.br
UFBA, Bahia, Brazil
UEFS, Bahia, Brazil

Tássio Virgínio
tassio.virginio@ifto.edu.br
UFBA, Bahia, Brazil
IFTO, Bahia, Brazil

Adriana Cruz
adriana.cruz@estudante.ufla.br
UFLA, MG, Brazil

Heitor Costa
heitor@ufla.br
UFLA, MG, Brazil

Ivan Machado
ivan.machado@ufba.br
UFBA, Bahia, Brazil

ABSTRACT

Test smells are fragments of code that can affect the comprehensibility and the maintainability of the test code. Preventing, detecting, and correcting test smells are tasks that may require a lot of effort, and might not scale to large-sized projects when carried out manually. Currently, there are many tools available to support test smells detection. However, they usually do not provide neither a user-friendly interface nor automated support for refactoring the test code to remove test smells. In this work, we propose RAIDE, an open-source and IDE-integrated tool. RAIDE assists testers with an environment for automated detection of lines of code affected by test smells, as well as a semi-automated refactoring for Java projects using the JUnit framework.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; *Empirical software validation*.

KEYWORDS

Test Smells, Unit Test, Test Refactoring, Automated Test

ACM Reference Format:

Railana Santana, Luana Martins, Larissa Rocha, Tássio Virgínio, Adriana Cruz, Heitor Costa, and Ivan Machado. 2020. RAIDE: a tool for Assertion Roulette and Duplicate Assert identification and refactoring. In *34th Brazilian Symposium on Software Engineering (SBES '20)*, October 21–23, 2020, Natal, Brazil. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3422392.3422510>

1 INTRODUCTION

Test smells are test code anti-patterns that correspond to bad design or implementation choices, which can affect software comprehensibility and maintainability [15]. Manual analysis of test smells, such as prevention, detection, and correction, may not apply to large systems. Additionally, test smell analysis is still challenging

as testers need to explicitly identify where in the test code the test smells are presented, so as they can be refactored.

There are few techniques and tools in the literature to support automatized test smells analysis and improve test code quality. Usually, existing tools only provide test smells detection and focus on showing either the number of existing test smells or whether a class is affected or not by a particular test smell. Those features may not be enough to support test smells correction, such a test code refactoring.

In a recent study, Garousi et al. [5] conducted a systematic literature review and presented a catalog of test smells and a summary of guidelines, techniques, and tools used to deal with the test code. Although they found 24 tools, only 8 out of them are available for download, as follows: TRex [18], TeReDetect [9], TestLint [13], TestHound [6], OraclePolish [7], TeCReVis [8], Similar Test Method Detector [2], and TestQ [1]. In addition to them, tsDetect¹ [11] and JNose Test² [16] were recently proposed to support test analysis.

The existing test smells detection tools for Java projects, with unit tests developed using JUnit are limited to inform either the presence or the absence of test smells and do not indicate their location in the test code. Although Van Deursen et al. [15] enlist a set of refactoring solutions to remove test smells, existing tool support do not provide automated mechanisms to fix them. In addition, there are no IDE-integrated test smells detection tools, and they commonly do not present a user-friendly interface.

Considering the aforementioned limitations of existing tool support, we proposed RAIDE³, a plugin for the IDE Eclipse that assists testers in identifying and refactoring test smells. RAIDE provides an automated detection of test smells and a semi-automated refactoring of the test code affected by them. RAIDE deals with Java projects and test code implemented with the JUnit framework. The current version of RAIDE detects two types of test smells (*Assertion Roulette* and *Duplicate Assert*) and refactors the test code related to them.

Furthermore, RAIDE integrates a conceptual framework under development by an inter-institutional researchers' team, which consists of test smells prevention, identification, refactoring, and visualization to improve the test code quality. The JNose Test and the TSVizzEvolution⁴ tools are also part of this framework.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBES '20, October 21–23, 2020, Natal, Brazil

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-8753-8/20/09...\$15.00

<https://doi.org/10.1145/3422392.3422510>

¹ Available at <https://testsmells.github.io/>

² Available at <https://github.com/tassiovirginio/jnose>

³ Available at <https://raideplugin.github.io/RAIDE/>

⁴ Available in <https://github.com/AdrianaPriscilaSantos/TSVizzEvolution>

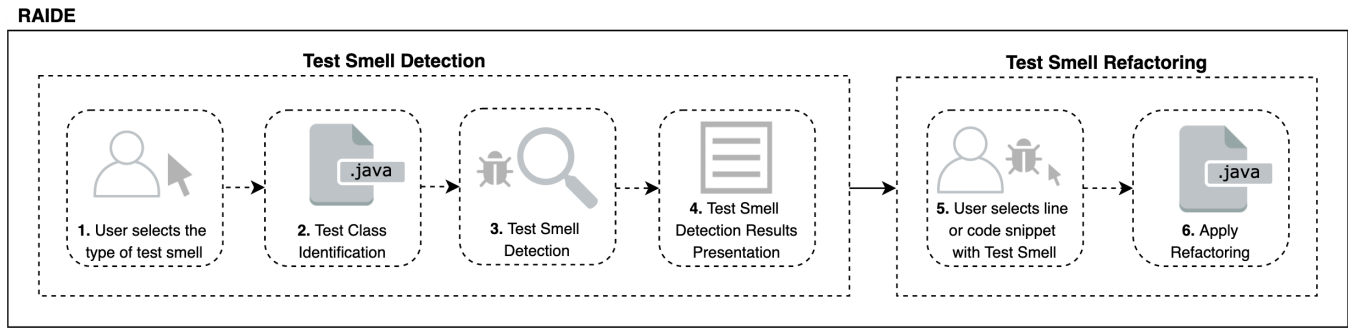


Figure 1: RAIDE overview

The remainder of this paper is structured as follows. Section 2 presents an overview of the RAIDE tool, and its architecture. Section 3 illustrates an example of use. Section 4 compares RAIDE with similar tools. Section 5 draws final remarks and points opportunities for future work.

2 RAIDE TOOL

The RAIDE⁵ tool is a plugin for the Eclipse IDE, and was built with the aim to detect test smells and refactor test code written with the JUnit framework in Java projects. For the detection of test smells, RAIDE reuses detection rules of the tsDetect tool, and for refactoring smelly test code, it reuses JDeodorant tool graphic components. RAIDE identifies test classes, methods, and lines affected by a specific test smell, which assist developers to check the test code against test patterns and improve its quality by refactoring solutions.

Although there are many test smells for different purposes, recent studies show that some test smells are more frequent than others. Currently, RAIDE tool supports the detection and refactoring of two types of test smells, *Assertion Roulette* (AR) and *Duplicate Assert* (DA). AR and DA are test smells related to the assertion structure. AR is one of the most frequent type of test smells in test code [10, 12], and duplicate-related test smells, such as DA, have been frequently discussed by academic community and gray literature [4].

The AR test smell occurs when the optional parameter for explaining the assertion structure is absent [15]. This parameter helps understand the purpose of the assertion and makes it easier to analyze why the method has failed. The AR test smell may make it difficult to identify which assertion produces an error in a test failure. To refactor the AR test smell, it is necessary to add an explanation parameter to the assertion structure. The DA test smell occurs when the test method has two or more assertion structures with the same parameters [11]. The DA hinders the purpose of the test method and violates the responsibility of each method to fulfill a single objective. If it is necessary to repeat the assertion with the same parameters to test different conditions, then a new test method should be created to test each one of them.

Figure 1 shows an overview of how the RAIDE tool works. It has two main processes and a set of steps, as follows: (i) **Test Smell**

Detection, which detects tests smells and presents them in the Eclipse IDE preview tabs (Steps 1 to 4); and (ii) **Test Smell Refactoring**, which encompasses a semi-automated test code refactoring approach to help developers fix the smelly test code (Steps 5 and 6). We better describe these processes and steps in the next sections.

2.1 Test Smell Detection

For the **Test Smell Detection** process (Figure 1), the **user selects the type of test smell** to be analyzed, either AR or DA (Step 1). After selecting the type, the tool performs a **Test Class Identification** (Step 2) to identify the project test classes and create an AST (Abstract Syntax Tree) for each test class. Then, the tool extracts information about the code structure through the AST and applies the detection rules for **Test Smells Detection** (Step 3). We reused detection rules from the tsDetect tool and performed improvements on them. tsDetect is a test smell detection tool available for command line on the terminal. It detects whether a test smell is either present or absent in a test class. In turn, RAIDE is integrated with an IDE and indicates the exact location (lines of code) of test smells. Next, we describe the detection rules for each test smell.

Assertion Roulette (AR). RAIDE searches for assertions structures (e.g., `assertTrue`, `assertNull`, `assertEquals`) in each test method of a given test class to count the number of parameters and check whether the optional parameter is missing. According to the number of parameters, the assertions structures are classified into 3 categories: **Category A:** structures with 2 mandatory parameters and 1 optional - `assertArrayEquals`, `assertEquals`, `assertNotSame`, `assertSame`, `assertThrows`, `assertThat`; **Category B:** structures with 1 mandatory parameter and 1 optional - `assertFalse`, `assertNotNull`, `assertNull`, `assertTrue`; and **Category C:** structures with 1 optional parameter - `fail`. For example, if an assertion in the **Category A** has less than 3 parameters, it means that the optional string parameter is missing; therefore, it is a candidate for AR. Similarly, if an assertion in the **Category B** has less than 2 parameters and the **Category C** has less than 1 parameter, they also contain AR test smells. In addition, we have extended the tsDetect's detection rules to check the content of the optional parameter, if any. RAIDE also considers an empty string ("") or strings with space (" ") as an AR.

⁵RAIDE tool is under the GNU General Public License.

Algorithm 1 Pseudocode to identify the AR

```

1: function IsAssertionRoulette(Assertion)
2:   CategoryA ← [assertArrayEquals, assertEquals, assertNotSame, ...]
3:   CategoryB ← [assertFalse, assertNotNull, assertNull, assertTrue]
4:   CategoryC ← fail
5:   Name ← name of Assertion
6:   if Name is contained in CategoryA then
7:     if (Assertion has ParametersSize < 3) or ExplanationIsEmpty(Assertion) then
8:       return true
9:     end if
10:  end if
11:  if Name is contained in CategoryB then
12:    if (Assertion has ParametersSize < 2) or ExplanationIsEmpty(Assertion) then
13:      return true
14:    end if
15:  end if
16:  if Name equals CategoryC then
17:    if (Assertion has ParametersSize < 1) or ExplanationIsEmpty(Assertion) then
18:      return true
19:    end if
20:  end if
21: end function

```

Algorithm 1 explains the AR identification. After identifying all the assertions of a method, each assertion is analyzed individually to check its category (lines 6, 11 and 16). Then, the number of parameters of an assertion is checked against the number of expected parameters according to its category (lines 7, 12 and 17). If the amount of parameters of the assertion under analysis is less than the amount of the expected parameters of the category, then it is likely that the optional parameter is missing (lines 7, 12 and 17). Even if no parameters are missing, the tool checks whether the string content presents an explanation (lines 7, 12 and 17) for the assertion. Therefore, if either the number of parameters is less than the number of expected parameters or the string is empty, RAIDE returns true (lines 8, 13 and 18) and classifies the assertion as an AR test smell.

Duplicate Assert (DA). Similarly to the AR analysis, RAIDE also searches for assertion structures to detect DA test smells. After identifying such structures, RAIDE compares them to identify assertions with the same parameters. If two assertion structures within the same method contain identical parameters, we consider them as a DA.

Algorithm 2 explains the DA identification method. It creates the variable *Size* and assigns this variable with the size of the list of assertions (line 2). The value of *Size* is used to create the vector *Checked[]* and to control the repeating structures that scan the entire list of assertions (line 3). It also creates the *Duplications[]*, a dynamic list to report the duplication in the method (line 4). An assertion can be repeated multiple times. Therefore, we assume that each set of assertions with equal parameters corresponds to a duplication. A test method is likely to have more than one duplication, i.e., multiple sets of duplicate assertions. To meet this scenario, we group the results of each duplication into a position in the *Duplications[]* list (line 22). Therefore, the algorithm goes through the entire list of assertions (line 5) and makes comparisons with other assertions that have not been “verified” as a duplication (lines 9, 10 and 11). If a duplication is identified, then the value *true* is assigned to the variable *DA* (line 12). The variable *Lines* is used to capture original *assertion* line (line 13), and *Checked[]* in position *i* receives the value *true* (line 14). After that, the duplicated assertion is concatenated and inserted in *Duplications[]* list. This process is repeated until an assertion is compared with all other assertions.

Algorithm 2 Pseudocode to identify the test smell DA

```

1: function IsDuplicateAssert(Assertions)
2:   Size ← size of Assertions
3:   let Checked[Size] be a new array
4:   let Duplications[] be a new array
5:   for i ← 0 to Size-1 do
6:     Lines ← empty string
7:     if not Checked[i] then
8:       DA ← false
9:       for j ← i+1 to Size-1 do
10:        if not Checked[j] & Assertions[i] == Assertions[j] then
11:          if not DA then
12:            DA ← true
13:            Lines ← Line(Assertions[i])
14:            Checked[i] ← true
15:          end if
16:          Lines ← Lines + ", " + Line(Assertions[j])
17:          Checked[j] ← true
18:        end if
19:      end for
20:    end if
21:    if DA then
22:      Duplications ← add Lines
23:    end if
24:  end for
25:  return Duplications
26: end function

```

After the test smells detection, RAIDE executes the **Test Smell Detection Results Presentation** (Step 4), which presents the IDE Eclipse tabs to show a list of test smells detected. To support this step, we reuse graphical components from the JDeodorant tool, such as the text area for presenting additional information about the test smells, and also the pointer for the test code location with the identified test smell highlighted in green color.

2.2 Test Smell Refactoring

To carry out the **Test Smell Refactoring** (Figure 1), the **User selects the line or code snippet with a test Smell** (Step 5), right after selecting it in the preview tab on Eclipse. RAIDE presents the location of the test smell in the test code, highlighting the affected line or code snippet with the green color. Besides, the tool also presents a definition for each test smell, i.e., an explanation to users, which might help them and avoid the insertion of new test smells.

To **Apply Refactoring** (Step 6), we create two strategies, one for each test smell (AR and DA). Next, we present them.

Assertion Roulette. RAIDE rewrites the structure of an assertion. We deal with two different situations. First, if there is no explanation in *assert*, the tool includes in the first parameter of the *assert* a repair string: “Add Assertion Explanation here”. Second, if the *assert* has an empty string or a string containing a space string, the tool rewrites that parameter with the same repair string: “Add Assertion Explanation here”. Since the repair string only indicates whether the tester should fix the test code, it is necessary that the tester rewrites the repair string with a more informative explanation. The editing of the test class file made by refactoring the RAIDE is manipulated with the help of the *IJavaElement* and *ITextEditor* interfaces, as well as the JDeodorant.

Algorithm 3 explains how the refactoring of test code with AR works. It receives an *Assertion* that should be fixed. The algorithm creates and instantiates a variable *Length* with the length of the assertion and a *Position* with the position of the first parenthesis of the assertion (lines 2 and 3). After that, the string *AssertionRefactored* is built. First, the *AssertionRefactored* receives a copy of the *Assertion* from the initial position “0” until *Position* value (line 4), concatenates the string “Add Assertion Explanation Here” (line 5),

Algorithm 3 Pseudocode to refactoring the test smell AR

```

1: function REFACTORINGASSERTIONROULETTE(Assertion)
2:   Length ← length of Assertion
3:   Position ← getFirstParenthesisPosition(assertion)
4:   AssertionRefactored ← copy(Assertion, 0, Position)
5:   AssertionRefactored+ ← ""AddAssertionExplanationhere,""
6:   AssertionRefactored+ ← copy(Assertion, Position + 1, Length)
7:   Replace(Assertion, AssertionRefactored)
8: end function

```

and concatenates a copy of the *Assertion* from *Position+1* until its last position (line 6). After that, the *Assertion* is replaced by the *AssertionRefactored* (line 7). To replace the assertion by the refactored one in the test class (line 7), RAIDE manipulates the test class through the interfaces *IJavaElement*, *ITextEditor*, *IDocumentProvider*, and *IDocument*, as well as the *JDeodorant*.

Duplicate Assert. RAIDE removes each duplicated assert of a method and places them in new methods, one for each duplicated assert. Then, the new methods are copies of the original one, but contain one single version of the duplicate assert. RAIDE preserves the original method as a comment, which might be used as a backup for the test engineer. The Algorithm 4 presents the pseudocode related to the refactoring of the DA test smell. The function receives two parameters: the method that contains the duplications (*Method*) and an array with the duplicate assertions of the method (*Assertions* in Figure 2), generated in the Algorithm 2. Before refactoring, the algorithm stores a copy of the original method (*CopyMethod* in line 2) and the original method commented (*BackupMethod* in line 3).

Each line (*array index*) of the *Assertions* array groups the duplications of the same assertion, as Figure 2 shows. For example, the array index 1 represents that a specific assertion is duplicated in lines 5 and 8 of the method. To refactor it, we convert this array in a new structure (*DuplicationOccurrence* array in line 4). The size of this new array is related to the amount of the method lines, in which each index corresponds to a line of the method (index 4 corresponds to the line 4 of the method). Hence, each array's position contains the occurrence of the assertion repetition. The value zero (0) (e.g., index 6 and 9) represents that either there is no assertion in the corresponding line of the method or there is an assertion, but it is not duplicated in that method. The value one (1) represents the first occurrence of the duplicated assertion, the value two (2) represents the second occurrence, and so on. For example, index 4 of the *DuplicationOccurrence* array (Figure 2) contains the value 1 to indicate that this is the first occurrence of a duplicate assert.

After creating the *DuplicationOccurrence* array (line 4), the algorithm refactors the original test method (*DeleteOtherAssertions()* in line 5) to remove the duplication. Thus, after refactoring, the method under analysis should only contain the first occurrence of each duplicated assertion. The algorithm creates copies of the method to include the extracted assertion (lines 8-10). Finally, the source code of the original method is replaced for: (i) the original method commented (for backup); (ii) the refactored method; (iii) and the extracted methods.

3 EXAMPLE OF USE

This section presents an example of use of the RAIDE tool. For the test smells detection, the user needs to select the option *Test Smells* in the top menu of the Eclipse IDE and select the type of test

Algorithm 4 Pseudocode to refactoring the test smell DA

```

1: function REFACTORINGDUPLICATEASSERT(Method, Assertions)
2:   CopyMethod ← Copy of Method
3:   BackupMethod ← Comment(CopyMethod)
4:   DuplicationOccurrence ← MapDuplications(Assertions)
5:   Method ← DeleteOtherAssertions(CopyMethod, 1, DuplicationOccurrence)
6:   Number ← HigherNumberOfDuplications(Assertions)
7:   NewMethods ← empty string
8:   for i ← 2 to Number do
9:     Extracted ← DeleteOtherAssertions(CopyMethod, i, DuplicationOccurrence)
10:    NewMethods ← add Extracted
11:  end for
12:  Replace(CopyMethod, BackupMethod + Method + NewMethods)
13: end function

```

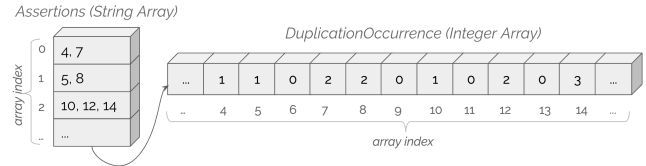


Figure 2: Description of the array conversion done by *MapDuplications()*

smell to analyze: either AR or DA (Step 1 in Figure 3). Then, the user selects the test package of the project under analysis (Step 2 in Figure 3), and clicks on the corresponding option to perform the identification of the test smell (Step 3 in Figure 3). For this example, we have selected the reflections project⁶, version 0.9.12, which is used for Java runtime metadata analysis.

After detecting the tests smells, the preview tabs show all smells detected in the test package, one test smell per line (Step 4 in Figure 3). When clicking on a reported test smell line, the tool highlights the affected source code line with a green color (Step 5 in Figure 3). When hovering the highlighted line, the test smell description is shown. Figures 4 and 6 show the descriptions of the AR and DA smells, respectively. Then, to refactoring the test code, it is necessary to double click on the reported test smell line in the preview tab and select the refactor option (Step 6 in Figure 3).

Figure 4 shows a code snippet with an AR smell. There are three undocumented *assertions* (lines 25-27) identified by the plugin RAIDE. Figure 5 shows the refactoring process for the second undocumented *assertion* (line 26). RAIDE added the first missing parameter, which included the following string, “Add Assertion Explanation here”. However, if the test method fails, the sequence generated by RAIDE will not correctly indicate which assertion is responsible for the failure of the test code, so to remove the test smell, the developer needs to rewrite the string generated by the RAIDE with the description that perfectly matches the verification made by the *assertion*.

Figure 6 shows a code snippet where two pairs of duplicate *assertions* were identified (lines 116 and 120; and lines 117 and 121). To refactor the test code regarding the *Duplicate Assert* test smell, RAIDE refactors the original method and extracts new methods. In addition, it preserves the original commented method as a backup for the test engineer. Figure 7 shows the resulting refactoring. The original test method (withReturn) was commented (lines 112-123) and two copies were made from the original method, method withReturn() (lines 126-134) and method

⁶Available at <https://github.com/ronmamo/reflections>

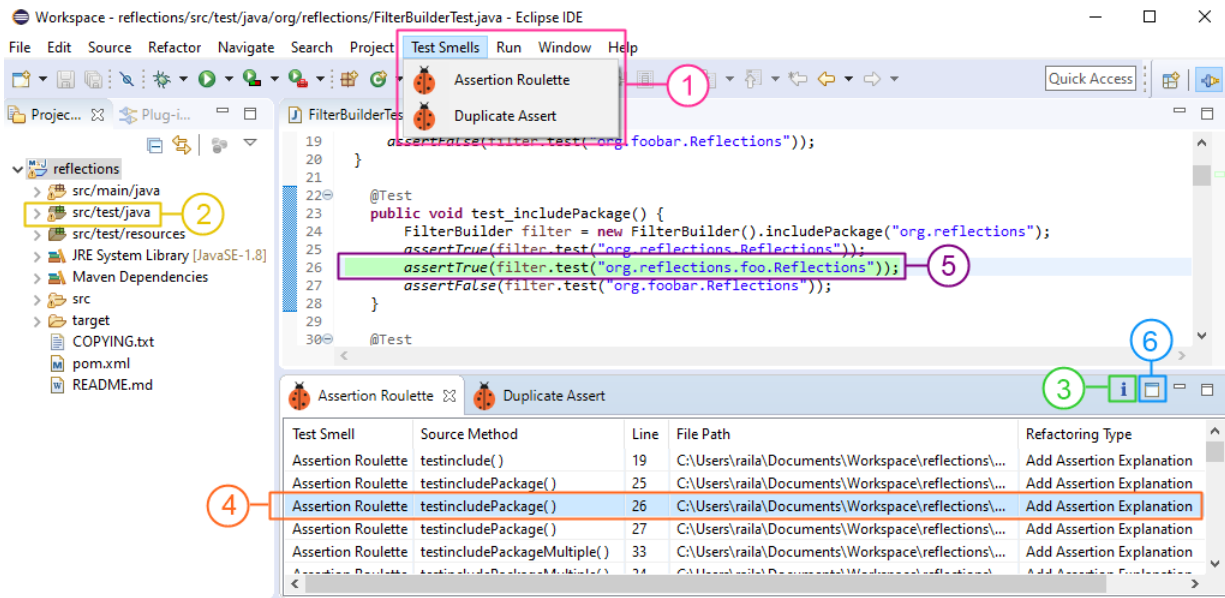


Figure 3: Steps to run the RAIDE

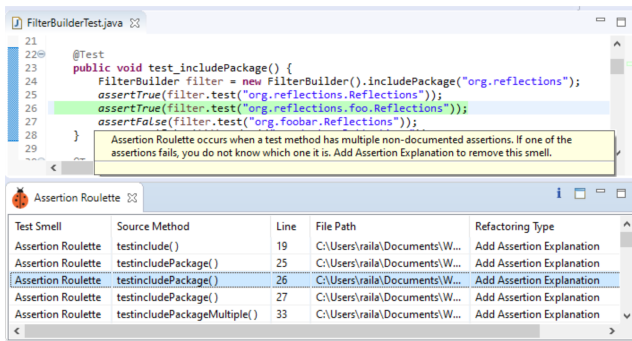


Figure 4: AR identified with the RAIDE plugin

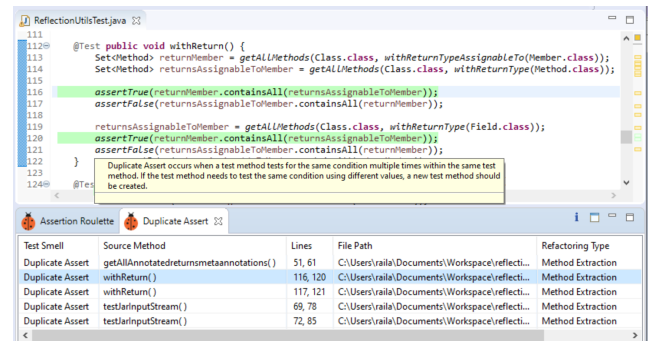


Figure 6: DA identified with the RAIDE plugin

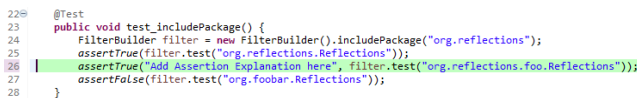


Figure 5: AR refactored with the RAIDE plugin

withReturnExtracted1() (lines 137-145). The first duplicate *assertion* pair (Figure 6, lines 116 and 117) was preserved in the original method (Figure 7, lines 130 and 131); and the second duplicate *assertion* pair (Figure 6, lines 120 and 121) was preserved in the second method, which was extracted, as Figure 7 shows in lines 143 and 144.

4 COMPARISON WITH SIMILAR TOOLS

The literature presents many tools to detect smells. For example, the JDeodorant⁷ is an Eclipse IDE plugin to identify code smells in Java projects and resolve them applying code refactorings. Initially, the first version of the tool only supported one smell, the *Feature*

⁷Available at <https://github.com/tsantalis/JDeodorant>



Figure 7: DA refactored with the RAIDE plugin

Envy [3, 14]. Currently, the tool supports four other types of code smells: *Type/State Checking*, *Long Method*, *God Class*, and *Duplicated*

Code. For each of them, there is a refactoring indicating which is done automatically by the tools. In the JDeodorant tool, code smells are detected using the ASTParser API to identify switch/if statements. To apply the refactorings, the plugin uses the ASTRewrite API. While JDeodorant aims to improve the quality of the production code, RAIDE assist tester to improve the quality of the test code.

tsDetect is a test smell detection tool available only through command line interface (CLI). It detects 21 types of test smells. The identification process carried out by tsDetect involves the execution of 3 tools⁸: (i) *Test File Detector*, detects the project's unit test files; (ii) *Test File Mapping* links the unit test files to the production files being tested; and (iii) *Test Smell Detector*, detects test smells in the test code. Each of these tools is executed separately following a predefined order. In addition, it is required to manipulate each file manually. At the end of the execution of the three tools, tsDetect generates a .cvs file with Boolean results to inform if each test class of the project under investigation contains (true) or not (false) test smells. Although tsDetect includes a large number of types of test smells, it is not IDE-integrated and do not present a straightforward way to present the results (.cvs file). Thus, finding the test smell on the code might be difficult and costly for the user.

The JNose Test tool⁹ provides an automated detection of test smells and collects metrics in the Java test code. JNose Test is an extension of the tsDetect tool and, therefore, it also analyzes 21 types of test smells. JNose Test integrates the three projects used by tsDetect in a single tool and has a graphical web interface, which include automated collection of code and coverage metrics. The tool can perform the analysis of several projects that are in a single directory, generating only one output file, which is also a .csv. However, the JNose Test file presents the amount of each type of test smells per class [16, 17]. Although JNose Test improves the testing process compared to the tsDetect, it is not integrated with an IDE for software development and also requires manual inspection and refactoring.

Similarly to RAIDE, the TRex¹⁰ tool is an Eclipse IDE plugin and provides test smells refactoring. TRex provides an infrastructure to refactor test smells for TTCN-3, a strongly typed testing language used in conformance testing of communicating systems. TRex calculates TTCN-3 metrics and suggests refactorings for the TTCN-3 test based on the calculated metrics. It includes the following refactoring types: *Rename*, *Inline Template*, *Inline Template Parameter*, and *Merge Template* [18]. Although TRex is able to refactor the test code, it does not include analysis and testing of test smells in Java projects with the JUnit framework.

5 CONCLUSION

In this study, we presented RAIDE, a IDE-integrated tool to identify test smells and refactor smelly test code. This is an initial attempt to provide automated support for refactoring test smells in Java projects with JUnit. RAIDE automatically identifies two types of test smells, *Assertion Roulette* e *Duplicate Assert*, and provides a semi-automated approach to refactor the test code. As future work,

we intend to include more types of test smells and support other IDE environments and programming languages.

ACKNOWLEDGMENTS

This research was partially funded by FAPESB grant JCB0060/2016, INES 2.0, CNPq grant 465614/ 2014-0, and CAPES - Finance Code 001.

REFERENCES

- [1] Manuel Breugelmans and Bart Van Rompaey. 2008. Testq: Exploring structural and maintenance characteristics of unit test suites. In *WASDeTT-1: 1st International Workshop on Advanced Software Development Tools and Techniques*. IEEE International Conference on Software Maintenance, Beijing, China, 4–15.
- [2] Zheng Fang. 2014. *Test clone detection via assertion fingerprints*. Master's thesis. University of Waterloo.
- [3] Marios Fokaefs, Nikolaos Tsantalos, and Alexander Chatzigeorgiou. 2007. Jdeodorant: Identification and removal of feature envy bad smells. In *2007 IEEE International Conference on Software Maintenance*. IEEE, Paris, France, 519–520.
- [4] Vahid Garousi and Barış Küçük. 2018. Smells in software test code: A survey of knowledge in industry and academia. *Journal of systems and software* 138 (2018), 52–81.
- [5] Vahid Garousi, Barış Kucuk, and Michael Felderer. 2018. What we know about smells in software test code. *IEEE Software* 36, 3 (2018), 61–73.
- [6] M. Greiler, A. van Deursen, and M. Storey. 2013. Automated Detection of Test Fixture Strategies and Smells. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*. IEEE, Luxembourg, Luxembourg, 322–331. <https://doi.org/10.1109/ICST.2013.45>
- [7] Chen Huo and James Clause. 2014. Improving oracle quality by detecting brittle assertions and unused inputs in tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, Hong Kong, China, 621–631.
- [8] Negar Koochakzadeh and Vahid Garousi. 2010. TeCreVis: A Tool for Test Coverage and Test Redundancy Visualization. In *Testing – Practice and Research Techniques*, Leonardo Bottaci and Gordon Fraser (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 129–136.
- [9] Negar Koochakzadeh and Vahid Garousi. 2010. A tester-assisted methodology for test redundancy detection. *Advances in Software Engineering* 2010 (2010).
- [10] Fabio Palomba, Dario Di Nucci, Annibale Panichella, Rocco Oliveto, and Andrea De Lucia. 2016. On the diffusion of test smells in automatically generated test code: An empirical study. In *Proceedings of the 9th international workshop on search-based software testing*. ACM, IEEE, Austin, TX, USA, 5–14.
- [11] Anthony Peruma, Khalid Almalki, Christian D. Newman, Mohamed Wiem Mkaouer, Ali Ouni, and Fabio Palomba. 2019. On the Distribution of Test Smells in Open Source Android Applications: An Exploratory Study. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering (CASCON '19)*. IBM Corp., USA, 193–202.
- [12] Anthony Shehan Ayam Peruma. 2018. *What the Smell? An Empirical Investigation on the Distribution and Severity of Test Smells in Open Source Android Applications*. Ph.D. Thesis. Rochester Institute of Technology, Rochester, New York.
- [13] Stefan Reichhart, Tudor Girba, and Stéphane Ducasse. 2007. Rule-based Assessment of Test Quality. *J. Object Technol.* 6, 9 (2007), 231–251.
- [14] Nikolaos Tsantalos and Alexander Chatzigeorgiou. 2009. Identification of move method refactoring opportunities. *IEEE Transactions on Software Engineering* 35, 3 (2009), 347–367.
- [15] Arie Van Deursen, Leon Moonen, Alex Van Den Bergh, and Gerard Kok. 2001. Refactoring test code. In *Proceedings of the 2nd international conference on extreme programming and flexible processes in software engineering (XP2001)*. CWI (Centre for Mathematics and Computer Science), NLD, 92–95.
- [16] Tássio Virginio, Railana Santana, Luana Almeida Martins, Larissa Rocha Soares, Heitor Costa, and Ivan Machado. 2019. On the Influence of Test Smells on Test Coverage. In *Proceedings of the XXXIII Brazilian Symposium on Software Engineering (SBES 2019)*. Association for Computing Machinery, New York, NY, USA, 467–471.
- [17] Tássio Virginio, Luana Martins, Larissa Rocha Soares, Santana Railana, Heitor Costa, and Ivan Machado. 2020. An empirical study of automatically-generated tests from the perspective of test smells. In *Proceedings of the XXXIV Brazilian Symposium on Software Engineering (SBES 2020)*. ACM, New York, NY, USA, 5.
- [18] Benjamin Zeiß. 2006. A refactoring tool for TTCN-3. *Master's thesis, Institute for Informatics, ZFI-BM-2006-05, ISSN* (2006), 1612–6793.

⁸<https://testsmells.github.io/>

⁹Available at <https://github.com/tassiovirginio/jnose>

¹⁰Available at <http://www.trex.informatik.uni-goettingen.de/trac>